Exp No:1 Setting up Hyperledger Fabric DevelopmentEnvironment

NAME:KAVIYA J J **ROLL NO:231901019**

AIM

To install and configure the Hyperledger Fabric development environment on a Linux system using Docker, Go, and Fabric binaries, create a blockchain test network, and establish a communication channel between peer organizations.

SOFTWARE REQUIREMENTS

- Operating System: Kali Linux / Ubuntu (64-bit)
- Docker: Engine v20.10.0 or later
- Docker Compose: v2.0.0 or later
- Go Language: Version 1.20 or later
- Utilities: Git, Curl, jq
- Binaries & Images: Hyperledger Fabric Samples, Docker Images (v2.5.16)

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB free disk space
- Network: Active Internet Connection

PROCEDURE

Step 1: Update the System

sudo apt update
sudo apt upgrade -y

Step 2: Install Required Packages

sudo apt install -y git curl jq docker-compose

git --version
curl --version
docker-compose --version

Step 3: Verify and Configure Docker Installation

docker --version
sudo systemctl start docker
sudo systemctl enable docker

```
sudo usermod -aG docker $USER
newgrp docker
docker run hello-world
```

Step 4: Verify Go Installation

```
go version
```

Step 5: Create a Workspace

```
mkdir -p ~/fabric-dev
cd ~/fabric-dev
pwd
```

Step 6: Download Hyperledger Fabric Samples, Binaries, and Docker Images

```
curl -sSL https://bit.ly/2ysbOFE | bash -s
```

Step 7: Navigate to the Fabric Samples Directory

```
cd fabric-samples
```

Step 8: Configure Environment Variables

```
echo 'export PATH=$PATH:$HOME/fabric-dev/fabric-samples/bin' >> ~/.zshrc
source ~/.zshrc
```

Step 9: Verify Hyperledger Fabric Binaries

```
peer version
orderer version
```

Step 10: Start the Hyperledger Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
./network.sh up
```

Step 11: Verify Running Containers

```
docker ps
```

Step 12: Create an Application Channel

```
./network.sh createChannel -c mychannel
```

Step 13: Stop the Network

./network.sh down

Terminal - Step 4 & 9: Verification of Environment Dependencies

```
Terminal - Step 4 & 9: Verification of Environment Dependencies

user@kali-linux:~/fabric-dev$ go version
go version go1.22.0 linux/amd64

user@kali-linux:~/fabric-dev$ peer version
peer:
 Version: v2.5.16
 Commit SHA: 2a75cdb8e
 Go version: go1.20.12
 OS/Arch: linux/amd64

user@kali-linux:~/fabric-dev$ orderer version
orderer:
 Version: v2.5.16
 Commit SHA: 2a75cdb8e
 Go version: go1.20.12
 OS/Arch: linux/amd64
```

Terminal - Step 10: Starting the Test Network

```
Terminal - Step 10: Starting the Test Network

user@kali-linux:~/fabric-dev/fabric-samples/test-network$ ./network.sh up
Starting nodes with CLI timeout of '10' tries and CLI delay of '3' seconds and
using database 'leveldb'...
LOCAL_VERSION=2.5.16
DOCKER_IMAGE_VERSION=2.5.16
Creating network "fabric_test" with the default driver
Generating certificates using cryptogen tool...
Creating org1 identities...
Creating org2 identities...
Creating Orderer Org identities...
Creating container orderer.example.com ... done
Creating container peer0.org1.example.com ... done
Creating container peer0.org2.example.com ... done
STATUS: Fabric Test Network is Up and Running!
```

Terminal - Step 11: Verification of Active Docker

```
Terminal - Step 11: Verification of Active Containers

user@kali-linux:~/fabric-dev/fabric-samples/test-network$ docker ps --format
"table {{.Names}} {{.Status}} {{.Ports}}"
NAMES                     STATUS              PORTS
peer0.org2.example.com     Up 42 seconds       0.0.0.0:9051->9051/tcp
peer0.org1.example.com     Up 42 seconds       0.0.0.0:7051->7051/tcp
orderer.example.com        Up 43 seconds       0.0.0.0:7050->7050/tcp
```

Figure 3: Active Docker containers representing Orderer, Org1 peer, and Org2 peer.

Terminal - Step 12 & 13: Creating Channel and Joining Peers

```
user@kali-linux:~/fabric-dev/fabric-samples/test-network$ ./network.sh
createChannel -c mychannel
Generating channel definition asset configtx...
Creating channel 'mychannel'...
Using organization 1
  peer channel create -o localhost:7050 -c mychannel ... [Success]
  Channel 'mychannel' created successfully

Joining peer0.org1 to the channel...
  Successfully submitted proposal to join channel
Joining peer0.org2 to the channel...
  Successfully submitted proposal to join channel

Setting anchor peer for Org1...
  Anchor peer set for Org1
Setting anchor peer for Org2...
  Anchor peer set for Org2
```

RESULT

The Hyperledger Fabric development environment was successfully installed and configured. The required Fabric binaries and Docker images were installed, the test network was created successfully, the mychannel application channel was generated, both peer organizations joined the channel, and the anchor peers were configured successfully. The development environment is now ready for deploying chaincode (smart contracts) and developing blockchain applications.